

Investigating Test Smells in JavaScript Test Code

Dalton N. Jorge
Federal University of Campina
Grande (UFCG)
Campina Grande, PB, Brazil
daltonjorge@copin.ufcg.edu.br

Patrícia D. L. Machado
Federal University of Campina
Grande (UFCG)
Campina Grande, PB, Brazil
patricia@computacao.ufcg.edu.br

Wilkerson L. Andrade
Federal University of Campina
Grande (UFCG)
Campina Grande, PB, Brazil
wilkerson@computacao.ufcg.edu.br

ABSTRACT

Writing automated test cases is a challenging and demanding activity. The test case itself is software that requires proper design to ensure it can be implemented and maintained as long as the production code evolves. Like code smells, test smells may indicate violations of principles that negatively affect the quality of test code design, making it difficult to comprehend and, consequently, impairing its proper use and evolution. This work aims to investigate the occurrence of test smells in JavaScript test code and whether their presence can be correlated with test code quality. We perform an empirical study using the STEEL tool where the test suites of 11 open-source JavaScript projects from the Github repository are analyzed to detect a set of previously cataloged test smells. We then investigate: i) which ones occur more frequently; ii) whether given test smells are likely to occur together, and iii) if the presence of certain test smells is related to classical bad design indicators on the test code. We found that the *Duplicate Assert*, *Magic Number Test*, *Unknown Test* and *Conditional Test Logic* smells are the most common in JavaScript test code, whereas the *Mystery Guest*, *Ignored Test* and *Resource Optimism* smells are the least common. Moreover, the *Conditional Test Logic*, *Magic Number Test*, *Duplicate Assert* and the *Exception Handling* smells may often appear together. Furthermore, there is a moderate to a strong positive correlation between some smells count and quality measures in the test code. We can conclude that test smells are frequently found in JavaScript test code, and their presence may be an indicator of low design quality.

KEYWORDS

test smells, javascript, quality metrics

ACM Reference Format:

Dalton N. Jorge, Patrícia D. L. Machado, and Wilkerson L. Andrade. 2021. Investigating Test Smells in JavaScript Test Code. In *Brazilian Symposium on Systematic and Automated Software Testing (SAST'21)*, September 27-October 1, 2021, Joinville, Brazil. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3482909.3482915>

1 INTRODUCTION

Test automation, especially at the unit testing level, has been an extensive practice in the industry. The goal is to improve testing

effectiveness and reliability and reduce the costs of repeated executions. On the one hand, despite the advent of test case generation tools, manually writing test cases is still a prevalent activity due to limitations of current tools and also due to development practices such as test-driven development. On the other hand, writing automated test code is a tedious and highly error-prone task. Moreover, poorly designed test code can become a burden to the development team [19, 22].

A *test smell* is an anti-pattern in a test code that may negatively affect its comprehension and evolution, making it less valuable to support code production and regression testing [3, 6, 23]. Both academy and industry have reported their occurrences in test code and suggest that they should be addressed as part of a good test design practice since they can make test code evolution difficult and even lead to test code bugs or less efficiency in fault detection [19]. In this sense, several smells, guidelines, and tools have been investigated recently to prevent, detect, and fix test smells such as those presented in the survey performed by Garousi and Küçük [6] that collects knowledge from both grey and scientific literature.

Empirical studies have already shown that test code commonly has test smells. Moreover, there may be a relationship between test smells and low-quality indicators on both production and test code. For instance, Bavota et al. [3] show that test smells are widely spread in the JUnit test suites of the systems investigated in their study, and also most of the test smells considered may have a negative impact on code comprehension. Additionally, Spadini et al. [19] performed an extensive study with 221 major releases of ten software systems and found out that JUnit tests with smells are more change- and defect-prone and production code is more defect-prone when tested by smelly tests. Furthermore, a study performed by Tufano et al. [21] shows that developers have a low perception of test smells. Therefore, tools for automatically detecting them are needed.

Different languages and test frameworks may present different challenges to test design. Even though similar smells can be found in different languages, their effect may be different and specific smells may emerge in specific languages. For instance, JavaScript, a currently active programming language with an enthusiastic community, particularly in the scope of Web application development, presents many challenges for testers due to its highly dynamic nature [5, 12]. Moreover, differently from Java, there are several test frameworks for supporting unit testing in JavaScript such as, Mocha and Jest with no accepted standard. This fact can make it even harder to established test design practices. However, to the best of our knowledge, no studies have been presented in the scientific literature regarding the investigation of test smells in JavaScript test code. Moreover, Aljedaani et al. [1] reports as result of a recent systematic mapping study that there is still a lack of tool support for test smell detection in JavaScript. Overall, most

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAST'21, September 27-October 1, 2021, Joinville, Brazil

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8503-9/21/09...\$15.00

<https://doi.org/10.1145/3482909.3482915>

empirical results in the test smells area have been performed in the scope of JUnit and XUnit frameworks [6].

The goal of this work is to investigate the occurrence of test smells in JavaScript test code and whether their presence can be correlated with test code design bad practices. To perform the study, we collected 11 JavaScript programs from the Github repository and analyzed then according to a set of previously cataloged test smells. Due to the lack of a test smell detection tool in the JavaScript language, we developed STEEL to help us detect them automatically. Based on data collected, we analyzed: i) which test smells occur more frequently; ii) whether given test smells are likely to occur together, and iii) if the presence of certain test smells is related to code quality metrics. Studies in the literature have already investigated these issues regarding other programming languages such as Java. However, programming languages differ in their essence and applicability as well as in the community practices. Therefore, we cannot generalize results from Java studies to JavaScript. Finally, it is widely accepted that bad code design may impair test code quality. Thus, it is important to investigate whether bad programming practices may lead to the frequency of test smells. In summary, the main contributions of this paper are:

- A preliminary catalog of test smells presented in the literature that can be found in JavaScript test code;
- An investigation on the frequency of test smells in JavaScript code and a comparison with results presented in similar studies in the scope of the Java language;
- An investigation of the relationship between JavaScript test code quality and test smells.

This paper is structured as follows. Section 2 introduces a catalog of the test smells considered in this work and describes some code metrics adopted. Section 3 presents a motivational example with a real test case discussing the impact of the smells on test case usage and evolution. Section 4 discusses related work on test smells in general, and code smells in JavaScript. Section 5 briefly describes the STEEL tool. Section 6 presents the design of the study and Section 7 presents and discusses the results. Finally, Section 8 presents conclusions and pointers to further works.

2 BACKGROUND

This section presents the test smells investigated and some code quality metrics adopted in this work.

2.1 Test Smells

Different definitions for the term *test smell* have been presented in the literature. Garousi et al. [6] unify them simply as: “a test smell refers to any symptom in test code that indicates a deeper problem”. Like code smells, test smells are usually not bugs and do not prevent test code from running. However, when not fixed, they can lead to several issues such as low maintainability, low effectiveness to detect faults or flakiness [3, 22]. Garousi et al. [6] present a catalog with 139 test smells identified in formal publications and also from the grey literature. They have classified the smells into eight categories. This section presents the test smells investigated in this work, organized according to these categories. We selected these smells from the ones investigated [14, 15, 20] that can be applied to JavaScript test code.

Test Semantic/Logic.

- *Assertion Roulette* (AR): a test method has multiple non-documented assertions, making it difficult to identify which one causes a failure;
- *Conditional Test Logic* (CT): a test method contains one or more control statements, whereas it should be simple and execute all statements;
- *Eager Test* (EaT): a test method invokes several methods of the production code, making it difficult to comprehend and maintain;
- *Lazy Test* (LT): multiple test methods call the same production method, which may introduce redundancy or inconsistencies among them.

Code Related.

- *Duplicate Assert* (DA): a test method that checks the same condition multiple times, while testing with different values often requires different test cases;
- *Magic Number Test* (MN): assert statements in a test method contains numeric literals as parameters that may not provide their meaning;
- *Redundant Print* (RP): a test method with print statements, but test cases are executed with little or no human intervention.

Issues in Test Steps.

- *Empty Test* (Emt): a test method with no executable statements that it is never executed;
- *Exception Handling* (ExT): a test method for which pass or failure depends on the production/test method throwing an exception that is captured by catch statements, instead of using the frameworks specific exception assertions;
- *Redundant Assertion* (RA): a test method contains assertions that are either always true or always false, making them useless;
- *Unknown Test* (UT): a test method that does not contain assertions, so all executions will pass unless they raise an exception.

Dependencies.

- *Mystery Guest* (MG): a test method that uses external resources that may cause stability or performance issues;
- *Resource Optimism* (RO): a test method assumes that a resource, such as a File, exists.

Test Execution.

- *Ignored Test* (IT): a test method is suppressed from running, which can create an additional overhead at compilation time or increase test code complexity;
- *Sleepy Test* (ST): a test that makes a thread to sleep, which can lead to undesirable behavior such as race conditions.

2.2 Code Quality Metrics

IEEE defines quality metrics as a direct quantitative measure or a function whose inputs are software data and whose output is a single numerical value that can be interpreted as the degree to which an item possesses a given attribute [10]. Table 1 presents the description of each metric adopted in this study.

Table 1: Quality Code Metrics

Metric	Description
Physical SLOC (PL)	Lines in the source code excluding comment lines.
Logical SLOC (LL)	Executable statements in the source code.
Cyclomatic Complexity (CC)	Number of linear independent paths.
Cyclomatic Density (CD)	Ratio between c. complexity and the number of executable statements.
Halstead Bugs (HB)	Number of bugs expected in the program.
Halstead Difficulty (HD)	Estimate the difficulty of the program.
Halstead Effort (HE)	Estimate the effort to develop or understand a program.
Halstead Length (HL)	Length of a program calculated by the sum of all operands and operators.
Halstead Time (HT)	Time to implement a program.
Halstead Vocabulary (HVoc)	Sum of distinct operands and operators of a program.
Halstead Volume (HVol)	How much information of the code is needed to understand it.
Maintainability (Ma)	Relative maintainability of the code.

SLOC is commonly used to measure the extension of a program. In addition, this metric helps to infer other metrics such as the quantity of effort, productivity, and maintainability of the program.

Cyclomatic Complexity [11] and Cyclomatic Density [7] are metrics based on the source code's amount of linearly independent paths. Both metrics are used to predict the complexity and maintainability of a system.

Halstead [9] presents metrics related to properties of a software and the relations between them such as: Bugs, Difficulty, Effort, Length, Time, Vocabulary, and Volume. These metrics depend on the implemented algorithm - more specifically, from elements like the operands and operators. However, they must be independent of execution on any specific platform.

Maintainability is based on the lines of code, cyclomatic complexity, and Halstead metrics. It indicates the program's capacity of being evolved by fixing bugs and extending its life cycle [24].

3 MOTIVATIONAL EXAMPLE

Listing 1 is a test case extracted from *Sails*, a framework for web application development. In this example, we will do a thorough evaluation to detect design problems in the test code. Initially, note that lines 7 and 8 have assertions that evaluate the *status* and *body* properties of the *err* object. However, none of the assertions have error messages for when the test fails. This absence of error messages as an argument when there are multiple assertions in the same test case characterizes the *Assertion Roulette* smell.

Still in the example of the test case in question, we can verify that line 4 raises an exception through the *throw* statement. This practice is not recommended in test code, as it adds unnecessary complexity. If the purpose of the test is to evaluate whether the production code will raise an exception or not, the developer should use the specific treatments of the adopted test library. This type of anti-pattern is called the *Exception Handling* smell.

```

1 it ('should return the expected error when something
  throws', function(done) {
2   var ERROR = 'oh no I forgot my keys';
3   sails.get('/ errors / 1', function(req, res) {
4     throw ERROR;
5   });
6   sails.request('GET / errors / 1', {}, function(err) {
7     assert.deepEqual(500, err.status);
8     assert.deepEqual(ERROR, err.body);
9     done();
10  });
11 });

```

Listing 1: Example from Sails project

Finally, it is possible to notice that the assertion in line 7 makes a comparison of the *err.status* property with the numeric literal 500. A beginner developer, or a novice in the development team, will have difficulties in recognizing the purpose of the assertion. This type of anti-pattern is called the *Magic Number Test* smell and has the consequence of impairing the understanding and maintenance of the test case.

In this example, we illustrate the design evaluation of the code, by identifying three types of smells present in the test case. This detection activity will increase in complexity if we expand the catalog of detected test smells types. We must also take into account that the example illustrates only a test case. The version of the project to which this example belongs has 18 test suites with 135 test cases in total. Performing the detection manually is a job that requires time and effort, especially in a language with the characteristics that JavaScript has. In addition, to date, we have not identified academic papers that explore the consequences of the occurrence of test smells in the context of the JavaScript language.

4 RELATED WORK

Testing JavaScript programs is an active research field. Several frameworks for writing automated test cases are available such as *Mocha*, *Jest*, *Tape*, *Tap*, and *Nodeunit*. Mirshokraie *et al.* [12] propose a framework for automatic generation of unit tests in JavaScript. Also, Fard and Mesbah [5] performed an empirical study to investigate JavaScript tests regarding their prevalence, quality, and shortcomings. They conclude that tests for the server-side of the applications are more prevalent and have higher quality. Tests written in frameworks have higher quality, and generally, tests lack proper coverage. Moreover, Gyimesi *et al.* [8] propose a Benchmark of JavaScript bugs to support empirical studies for assessing new testing techniques. Additionally, Saboury *et al.* [17] investigate *code smells* in JavaScript server-side applications. They performed a large scale study that analyzed several application versions and a survey with developers. They conclude that code smells are serious design problems that can impair the maintainability and reliability of JavaScript applications.

However, to the best of our knowledge, *test smells* in JavaScript have not been addressed yet in the literature [1]. Therefore, in the sequel, we discuss empirical studies on test smells in other languages such as Java, highlighting their general findings.

Bavota *et al.* [3] present two empirical studies on test smells in JUnit tests. The first study performed an empirical analysis of 18 systems to analyze the distribution of test smells. In contrast, the

second performed a controlled experiment with 20 master students to analyze whether test smells affect code comprehension. Results show that test smells are frequent and have a strong negative impact on comprehensibility. *Eager Test* and *Assertion Roulette* are the most common smells found in the study, and the authors highlight that *Eager Test* frequently occurs when *Lazy Test* is present, but not the other way around. The authors extend the studies to other systems in [4], with a total of 27 systems and three replications of the second study, including a group of industrial developers. The results are similar, but with an additional finding, that comprehension is 30% better when test smells are not present.

Tufano *et al.* [21] present two empirical studies to investigate developers' perception of test smells and to analyze occurrences of test smells in test code, how long they remain in the code and whether their presence is related to code smells. The study focuses on five types of test smells. Results show that developers do not always identify test smells and recognize their harm and test smells tend to be introduced when code is first committed and remain for a long period. Moreover, clean (with no smells) test classes usually tests clean production code. *Eager Test* and *Assertion Roulette* are often associated with smells in the production code.

Spadini *et al.* [19] investigated 221 major releases of ten software systems to search for a relation between test smells and the proneness of code to change and defect. They focused on six types of smells. They found out that JUnit tests with smells are more change-prone and defect-prone than tests with no smells. Also, the more test smells, the higher is this effect, especially for the *Eager Test*, *Assertion Roulette*, and *Indirect Testing* smells. Moreover, the production code is more defect-prone when tested by smelly tests. Additionally, Spadini *et al.* [20] investigated the severity rate of test smells by analyzing 1,500 open-source projects to identify severity thresholds. The thresholds were applied to tune their detection tool to minimize false positives. Then they surveyed with developers to validate such thresholds by indicating actions to be taken regarding refactoring and the perceived impact of the smell on the test suite's maintainability. As a result, *Eager Test* and *Assertion Roulette* were rated as low to medium impact, *Conditional Test Logic* and *Mystery Guest* as medium to high impact, and *Ignored Test*, *Resource Optimism* and *Sleepy Test* as high impact.

Our work differs from the above mentioned since we investigate diffuseness of test smells and their relation to test code quality metrics in the scope of the JavaScript language. This is the first study to provide empirical evidence of test smells in this scope.

5 TOOL

So far in our research, we have not found an existing tool for detecting test smells in the JavaScript language [1]. So we developed STEEL, a static analysis tool for the command line that allows the automatic detection of smells in test codes written in the JavaScript language. In addition to detecting test smells, the STEEL tool also reports test quality metrics. Among the metrics are: the number of physical and logical lines in the source code, the indexes of simple and condensed Cyclomatic Complexity, Maintainability, and Halstead properties (Bugs, Difficulty, Effort, Length, Time, Vocabulary, and Volume).

The architectural set of the STEEL tool consists of the following modules: the parser of the JavaScript language containing the lexical and syntactic analyzers, the rule-based test smells detectors, the code quality analyzer, and the report generator in JSON, HTML, and CLI formats. Figure 1 illustrates the referred STEEL architecture.

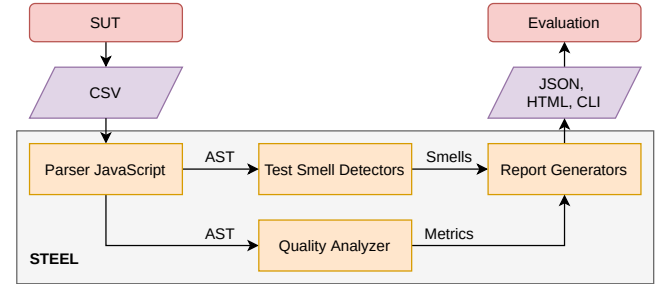


Figure 1: Tool Architecture

The flow of this architecture starts from SUT, in which the user of the tool must manually create a file of type CSV. This file will contain the paths of the source code files of the tests to be analyzed by the tool and, optionally, the paths of their respective production source code files. The obligation to declare the paths of the production source code files is only for the detection of smells of types *Eager Test* and *Lazy Test*. Then, the tool is executed by passing the CSV file as a parameter via the command line.

The first internal phase of the tool is the transformation of the test source code and the production source code (if informed) into AST (Abstract Syntactic Tree) structures. The second phase comprises two paths, the first of which aims to detect smells in AST by means of static analyzers specific to each type of smell. In the second way, AST is evaluated by the quality analysis module, where the metrics (number of lines of code, cyclomatic complexity/density, maintainability and *Halstead*) are extracted.

Finally, the results of the analysis of smells detection and extraction of quality metrics are used in the report generation phase. Detected smells and code complexity metrics are exported in JSON and HTML (Figure 2) formats. In addition, the results are also printed on the command line interface for evaluation and use in the continuous integration process.

6 EMPIRICAL STUDY: DEFINITION

In this work, we conducted an empirical study to understand the incidence and unfolding of test smells in the JavaScript language. For this purpose, we used the Goal, Question, Metric (GQM) [2] approach to define the study questions and interpret the data. We also adopted some guidelines for conducting the study [16]. We outline the goal of this study as follows:

*Analyze test smells
for the purpose of evaluating them
with respect to their incidence, interdependence and relation
to other code metrics
from the viewpoint of developers
in the context of open-source JavaScript projects.*

The study is essentially observational and non-comparative to related approaches for test smells detection since there is still a lack of studies investigating the incidence and impact of test smells in JavaScript test code as well as tools for automated detection in this context. Besides investigating smells occurrence, we also analyse whether occurrence can be correlated to standard code quality metrics in the test code in search of understanding the role and contribution of test smells in this context as test code quality indicators.

In the next subsections, we present the research questions that guide the study, the study design and the experimental objects.

6.1 Research Questions

Our study is designed to address the following research questions:

RQ1 *What are the most frequently detected test smells in JavaScript projects?* With this question, we intend to know which test smells are most frequent in the selected subjects by counting their occurrences. Also, this question serves as an indicator that STEEL fulfills its role as a detection tool for test smells.

RQ2 *Is there a significant association between the detected test smells?* In this question, we aim to find out whether given test smells are likely to occur together.

RQ3 *Is there a significant association between detected test smells and software quality metrics?* This last research question aims to find out if the presence of certain test smells is related to classical bad design indicators on the test code.

To answer RQ1, we verified the STEEL tool results to reveal which test smells appear more often along with the test suites of all projects. Once we identified and classified the occurrence of the smells, to answer RQ2, we analyzed the relationship between the test smells using Spearman's rank correlation coefficient. Likewise, to answer RQ3, we evaluate the interdependence of test smells and code quality metrics using Spearman's rank correlation coefficient.

6.2 Study Design

To minimize our research efforts and enable the reproducibility of our study, we develop an automated setup consisting of two processes (as seen in Figure 3).

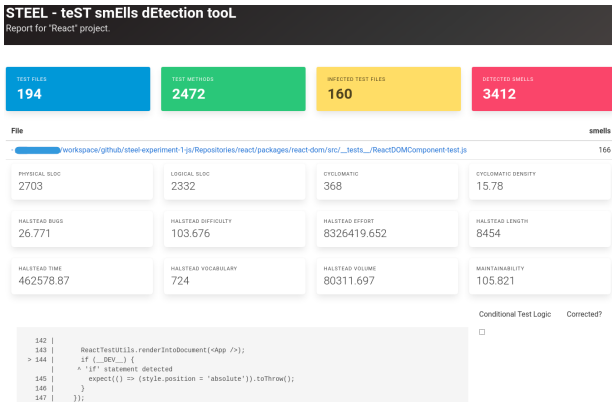


Figure 2: HTML report example from STEEL

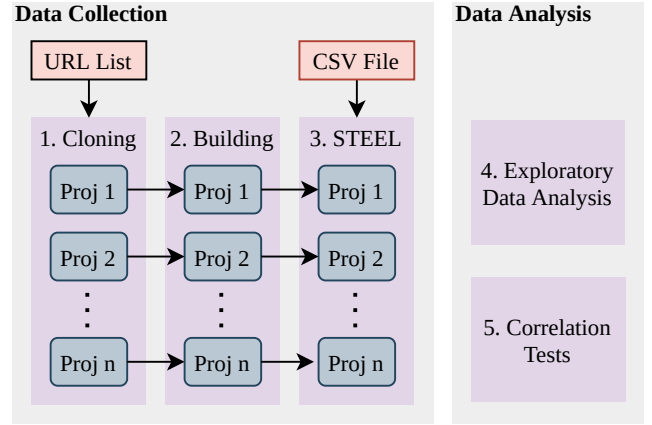


Figure 3: Study Design

The first process one is the *data collection*, automated with JavaScript code. It includes the following steps: (1) a list of repository URLs is provided for cloning specific versions of selected projects; (2) performing required procedures from each project for building unit tests environment; (3) executing the STEEL tool for each project providing a CSV file containing the unit test file paths.

The second process is the data analysis process, automated by R scripts. It consists of two steps: (4) performing an exploratory data analysis to reveal initial metrics and (5) computing correlation tests between detected test smells and code metrics.

6.3 Experimental Objects

To conduct this study, we selected eleven open-source software projects. Due to the diversity of projects available in public repositories, JavaScript dialects, and testing frameworks, we defined the following specific set of selection criteria for the projects:

- Hosted in the GitHub¹ repository;
- Encoded in JavaScript or Typescript languages;
- Having unit tests and use assertions from *Node.js* or *Chai* or *Jest*;
- Having more than 5000 stars in the ranking of *GitHub*;
- Having an active repository, with constant evolution.

Essentially, we focus on the Chai and Jest frameworks addressed by the STEEL tool. Moreover, we wish to explore apps that are more constantly used and are in evolution. In this context, constant modifications that often contribute to code degradation may also favor test smells introduction.

Table 2 presents the eleven projects listed for the study, containing for each project the version analyzed (*Version*) and the number of stars (*stars*) until the moment of analysis of the data. The following metrics can also be observed:

- # T.S.: number of test suites;
- # T.C.: number of test cases;
- # P.SLOC: total of physical lines of code;
- # L.SLOC: total of logical lines of code.

¹<https://www.github.com>

Table 2: Projects Data

Project	Version	Stars	# T.S.	# T.C.	P.SLOC	L.SLOC
babel	v7.6.0	39352	54	828	9090	8326
dayjs	v1.8.14	35638	12	122	661	1463
faker.js	v4.1.0	32816	17	182	2212	1943
hypernova	v2.5.0	5722	13	61	954	909
json-server	v0.15.0	55110	9	124	1433	1606
next.js	v8.1.0	70424	9	26	120	110
nuxt.js	v2.8.1	36948	92	733	9470	11367
react	v16.8.6	171232	194	2472	84221	64415
riot	v4.2.0	14510	2	49	723	783
sails	v1.2.2	21975	18	135	2730	2578
webpack	v4.35.0	58663	69	284	5652	5184

7 EMPIRICAL STUDY: RESULTS & ANALYSIS

In this section, we present and discuss the results² found in the study. Section 7.1 presents the data and answers the research questions while Section 7.2 highlights and discusses further important observations. Finally, Section 7.3 presents threats to validity.

7.1 Study Results

The research questions are addressed in this section as follows.

7.1.1 RQ1: What are the most frequently detected test smells in JavaScript projects? Table 3 presents a summary of the data collected in our study for test smell detection, where # Smells and # Smelled T.S. represent, for each project, the number of smells and the number of infected test suites respectively. We can see that test smells occurs in all projects having from 33% to 100% of the test suites infected. This demonstrates that even projects in constant evolution and with wide adoption and collaboration by the community - *React* and *Babel*, for example - are susceptible to the incidence of smells in their test suite base.

Table 3: Frequency of Smells by Project

Project	# T.S.	# Smells	# Smelled T.S.	% Smelled T.S.
babel	54	374	26	48.1
dayjs	12	40	5	41.7
faker.js	17	207	17	100.0
hypernova	13	88	8	61.5
json-server	9	115	6	66.7
next.js	9	11	3	33.3
nuxt.js	92	235	52	56.5
react	194	3412	160	82.5
riot	2	29	2	100.0
sails	18	280	15	83.3
webpack	69	424	32	46.4

Table 4 shows the measures of central tendency, where the values of means and medians are very different, indicating an asymmetric distribution. The table also presents the standard deviation dispersion measures where it is possible to observe different values and outliers. The property of *Skewness* reinforces the asymmetry of the

²<https://figshare.com/s/3488b1397f666d3a870f>

data with a tail to the right (positive values). The Shapiro test for normality reveals very low values for *p-values*, indicating that the data distribution is non-parametric.

Table 5 presents the number of occurrences of smells by type for each project. It can be noticed that some smells tend to occur more frequently such as the *Duplicate Assert*, *Magic Number Test*, *Unknown Test* and *Conditional Test Logic* smells. On the other hand, the least frequent ones are *Resource Optimism*, *Ignored Test* and *Mystery Guest*. However, the number of occurrences varies according to the project. Also, it is important to remark that no occurrence of the *Empty Test* smell was detected in any project.

Therefore, regarding RQ1, collected data show that test smells may be commonly found in JavaScript test suites and *Duplicate Assert* is the most frequent one.

7.1.2 RQ2: Is there a significant association between the detected test smells? The correlation test indicates whether a value changes in a variable when another variable changes its content. Thus, the correlation coefficient expresses the strength and the sense of the simultaneous variation of two variables. Another point, no less important, is the fact that the correlation does not imply causality but only an indication of a relationship between variables that need further study. In this work, we consider the correlation thresholds suggested by Salkind [18] to interpret the results, where coefficients between 0.4 and 0.6 indicate a *moderate* correlation, coefficients between 0.6 and 0.8 indicate a *strong* correlation, and coefficients greater than 0.8 mean a *very strong* correlation.

To investigate RQ2 and RQ3, we used the correlation test to assess whether the types of smells detected have any relationship with each other and between the quality metrics extracted from the source code of the tests. Note that we use the *Spearman* coefficient due to the non-normality of the data observed in the study.

Correlation test results between smells and quality metrics are presented in a heatmap in Figure 4. Since we found 0 occurrences of the *Empty Test* smell, it is not included in the heatmap.

We can observe strong positive correlations between the smells *Conditional Test Logic*, *Exception Handling* and *Magic Number Test* and also between the smells *Duplicate Assert*, *Exception Handling*, and *Magic Number Test*. In particular, there is a very strong correlation between the *Conditional Test Logic* and the *Magic Number Test* smells and between the *Duplicate Assert* and *Exception Handling* smells. Due to the complexity imposed by conditional statements and exceptional handling, we might expect to find assertion duplication and the use of magical numbers. Moreover, we often find exception handling along with conditional statements.

Additionally, we can find strong positive correlations between *Mystery Guest*, *Ignored Test*, and *Resource Optimism*. One possible cause of these correlations is that developers tend to disable the test case when test cases fail because there is no external resource. Moreover, there are very strong correlations between the *Assertion Roulette* and *Redundant Assertion* smells. Such smells may arise when the tester creates several assertions to enforce the test case's ability to fault detection and end up leaving them undocumented or missing their validity.

Overall, for any of the smells, it is possible to observe at least a moderate correlation to another smell, with coefficient ≥ 0.5 ,

Table 4: Basic Exploratory Data Analysis

Variable	S. Deviation	Skewness	mean	Q1	Median	Q3	Shapiro	p-value
Assertion Roulette	5.052398e+01	2.733220	2.254545e+01	0.000	0.000	15.500	0.535	0.0000040
Conditional Test Logic	1.778967e+02	2.752398	8.927273e+01	2.500	20.000	66.000	0.570	0.0000107
Cyclomatic Complexity	3.137402e+03	3.200007	1.415818e+03	121.500	374.000	1076.000	0.466	0.0000006
Cyclomatic Density	8.196744e+02	2.875044	5.146017e+02	77.870	295.934	538.026	0.594	0.0000215
Duplicate Asserts	3.000468e+02	3.286090	1.106364e+02	4.000	19.000	37.000	0.406	0.0000001
Eager Test	8.256127e+00	2.895915	3.181818e+00	0.000	0.000	0.000	0.463	0.0000006
Exception Handling	7.473213e+01	3.092516	3.209091e+01	0.000	1.000	29.000	0.491	0.0000012
Halstead Bugs	1.825686e+02	3.157865	8.390945e+01	9.247	20.366	55.438	0.480	0.0000009
Halstead Difficulty	2.342179e+03	3.129827	1.128323e+03	121.436	358.065	873.159	0.498	0.0000015
Halstead Effort	5.191004e+07	3.278480	1.936671e+07	1103301.910	2190316.262	8026758.479	0.408	0.0000001
Halstead Length	6.940229e+04	3.125038	3.306527e+04	4150.000	9091.000	23183.500	0.494	0.0000013
Halstead Time	2.883891e+06	3.278480	1.075928e+06	61294.551	121684.238	445931.027	0.408	0.0000001
Halstead Vocabulary	7.233952e+03	3.005825	3.855000e+03	501.500	1443.000	3270.000	0.540	0.0000046
Halstead Volume	5.477087e+05	3.157864	2.517276e+05	27736.796	61098.552	166309.985	0.480	0.0000009
Ignored Test	2.700168e+00	3.246311	9.090909e-01	0.000	0.000	0.000	0.396	0.0000001
Lazy Test	7.877471e+00	2.335396	3.363636e+00	0.000	0.000	0.000	0.508	0.0000019
Logical SLOC	1.872423e+04	3.112843	8.971273e+03	1186.000	1943.000	6755.000	0.498	0.0000014
Magic Number	2.199764e+02	3.209328	9.972727e+01	11.500	24.000	66.500	0.464	0.0000006
Maintainability	5.080052e+03	2.745302	3.294339e+03	591.661	1715.877	3234.021	0.621	0.0000459
Mystery Guest	3.045115e+00	2.529928	1.545455e+00	0.000	0.000	2.000	0.593	0.0000209
Physical SLOC	2.462437e+04	3.208463	1.066055e+04	838.500	2212.000	7371.000	0.458	0.0000005
Redundant Assertion	1.022564e+01	3.311475	3.181818e+00	0.000	0.000	0.000	0.359	0.0000000
Redundant Print	2.502544e+01	3.316625	7.545455e+00	0.000	0.000	0.000	0.345	0.0000000
Resource Optimism	1.206045e+00	3.316625	3.636364e-01	0.000	0.000	0.000	0.345	0.0000000
Sleepy Test	4.880387e+00	2.069351	2.727273e+00	0.000	0.000	2.500	0.647	0.0001001
Unknown Test	2.054824e+02	3.056188	9.700000e+01	8.500	21.000	67.500	0.512	0.0000021

Table 5: Detected Smells by Project

Project	AR	CT	DA	EaT	EmT	ExT	IT	LT	MN	MG	RA	RP	RO	ST	UT
babel	0	70	68	0	0	3	0	13	73	3	0	0	0	0	144
dayjs	0	1	27	0	0	1	0	0	10	0	0	0	0	0	1
faker.js	166	23	0	0	0	0	0	0	14	0	1	0	0	0	3
hypernova	31	3	6	8	0	0	0	0	24	0	0	0	0	2	14
json-server	0	5	0	0	0	0	0	0	13	1	0	0	0	3	93
next.js	0	2	2	0	0	0	0	0	5	0	0	0	0	0	2
nuxt.js	0	20	27	27	0	33	1	24	58	3	0	0	0	0	42
react	0	595	1013	0	0	253	0	0	757	0	0	83	0	9	702
riot	0	0	8	0	0	0	0	0	0	0	0	0	0	0	21
sails	51	62	19	0	0	38	0	0	60	0	34	0	0	1	15
webpack	0	201	47	0	0	25	9	0	83	10	0	0	4	15	30
Total	248	982	1217	35	0	353	10	37	1097	17	35	83	4	30	1067

showing that they may represent together some given test code flaw.

Therefore, regarding RQ2, we may conclude that certain test smells tend to occur together in the investigated JavaScript projects. The co-occurrences meaning may be explained by factors such as the nature of the smell or particularities of JavaScript test code. This is subject to further investigation.

7.1.3 RQ3: Is there a significant association between detected test smells and software quality metrics? To investigate RQ3, we consider again the heatmap presented in Figure 4. It shows that the *Conditional Test Logic* (CT) smell has very strong correlations (coefficients greater than 0.8) with all code quality metrics. This results indicates that CT adds a higher level of complexity to the test code by including decision and repetition structures.

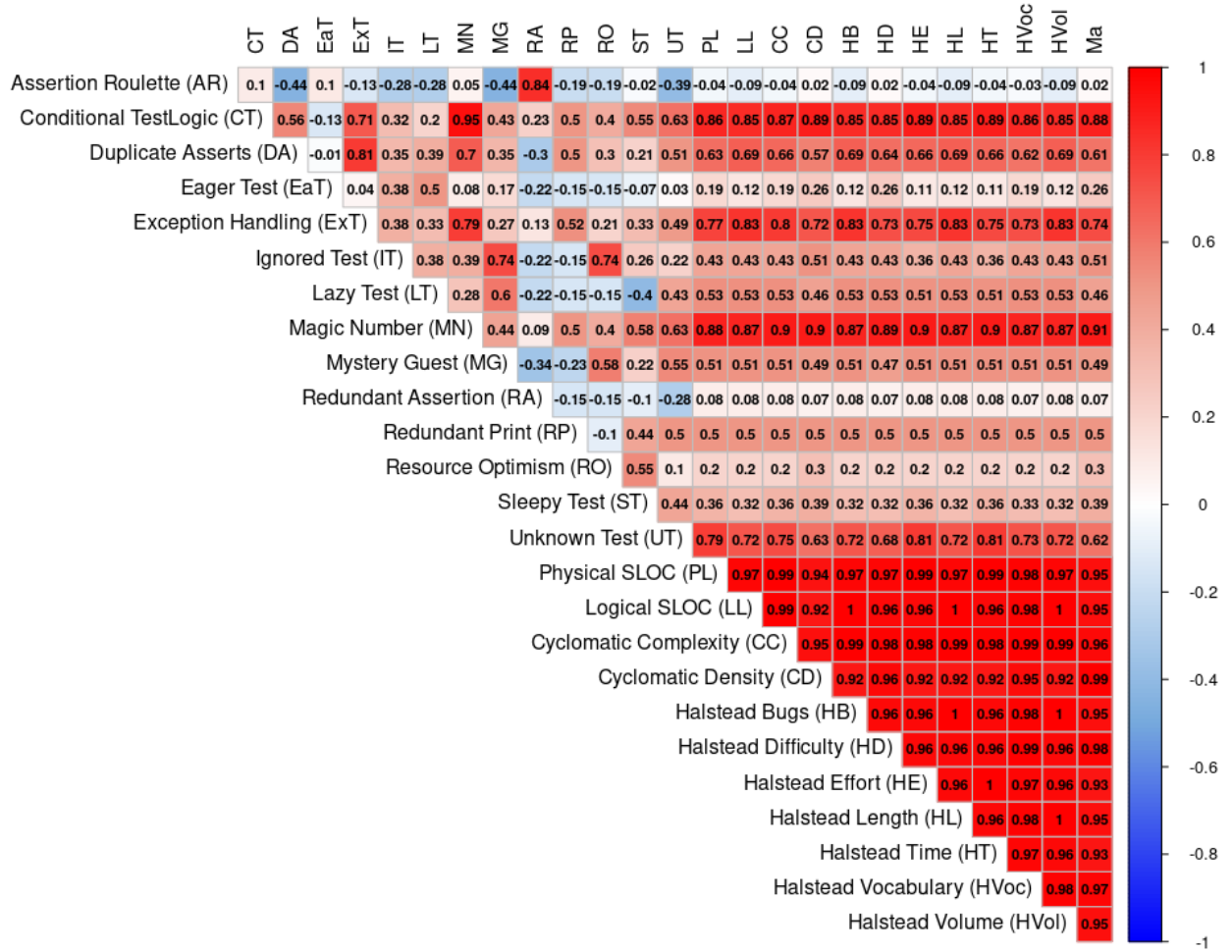


Figure 4: Correlation heat map for Metrics and Test Smells

We also found that the *Duplicate Assert* (DA) smell has strong correlations (coefficients between 0.6 and 0.8) with quality metrics, with the exception of Cyclomatic Density metric whom represents a moderate relationship (0.57). These values can be explained by the reason that DA causes repetition of blocks of code, impacting the total size of lines of code, in addition to increasing the complexity of understanding and maintenance.

The *Exception Handling* smell (ExT) has strong (coefficients between 0.6 and 0.8) and very strong (coefficients greater than 0.8) relationships with all quality metrics too. These correlations give evidence that ExT can increase: the number of physical and logical lines, the complexity, the chances of bugs occurring, the understanding of the code and the evolution of the test.

Also, the results show the *Magic Number Test* smell (MN) with very strong correlations (coefficients greater than 0.8) with all the quality metrics. Although the incidence of MN can increase the complexity of the code, increase the probability of bugs occurring, and decrease the readability and maintainability of the tests, we cannot say that the metrics indicate the smell or its effects.

The *Lazy Test* smell (LT) showed a moderate correlation (coefficients between 0.4 and 0.6) with all quality metrics, which seems to indicate that a scenario where the repetition of the *Eager Test* within the The same test suite increases the incidence of *Lazy Test*.

Table 6 shows the overall quality metrics computed for each of the projects. From Figure 4, we can see that, for the test code of the projects investigated, they are strongly correlated.

Regarding RQ3, we can observe that some smells have moderate to strong correlation to quality metrics where others have low or very low correlation. Also, some cases present very strong correlations. On one hand, correlation to quality metrics may suggest that test smells indeed indicate weaknesses in the test code as they occur along with standard code flaws. On the other hand, the lack of correlation may indicate that test smells represent a novel source of information to pinpoint test code specific flaws. A number of studies have been performed to investigate the severity and importance of test smells [4, 19]. Further research is still needed especially when considering JavaScript test code.

Table 6: Calculated Metrics by Projects

Project	PL	LL	CC	CD	HB	HD	HE	HL	HT	HVoc	HVol	Ma
babel	9090	8326	1188	440.264	68.798	957.299	12173509.23	28431	676306.068	3283	206392.343	2871.801
dayjs	661	1463	96	46.932	10.141	101.316	804021.53	4455	44667.862	472	30421.225	507.466
faker.js	2212	1943	374	357.205	20.366	405.849	2190316.26	9091	121684.238	1449	61098.552	1970.656
hypervnova	954	909	147	157.786	8.352	185.595	1256904.22	3845	69828.013	531	25052.366	944.323
json-server	1433	1606	247	93.451	15.334	141.557	1757010.51	5878	97611.696	820	46000.487	675.856
next.js	120	110	23	62.290	0.701	29.330	28774.22	383	1598.568	127	2100.757	363.637
nuxt.js	9470	11367	1189	654.558	100.348	1319.307	11232176.01	42288	624009.778	5658	301046.062	5583.135
react	84221	64415	10782	2889.553	626.746	8076.006	175378757.00	238793	9743264.282	25058	1880245.579	17789.975
riot	723	783	75	26.859	7.976	48.207	949699.60	3050	52761.089	307	23928.133	218.759
sails	2730	2578	489	295.934	22.165	358.065	2441289.12	9568	135627.174	1443	66490.934	1715.877
webpack	5652	5184	964	635.787	42.077	789.019	4821340.95	17936	267852.276	3257	126227.627	3596.241

7.2 Discussion

In this section, we further discuss the results by looking into the projects, the nature of the JavaScript language and test frameworks, reviewing results presented in other scopes.

7.2.1 Distribution of Test Smells and Test Code Style. In our study, the percentage of infected test suites varies from one system to another. Moreover, the occurrences of test smell vary both in numbers and the types of smells detected in the different systems. We could not find any particular feature of the systems that may explain in isolation these variations. In the study analysis, we considered the findings of each test suite independently of which of the systems it belongs to.

It is important to remark that we can observe a majority use of dialects from the frameworks Chai and Jest instead of the standard Node.js assert module in all of the test suites from the different systems. Therefore, our results represent the testing practice focused on those frameworks that promote more expressiveness, a readable style, flexibility, and accuracy on testing observations.

7.2.2 Test Smell Occurrences Compared to Other Studies. Several studies have investigated the occurrence and co-occurrence of test smells in test code, especially in the Java language. It is unreasonable to directly compare those studies to ours since the set of investigated smells often differ besides the programming language and the set of subjects. Nevertheless, it is important to relate common or different findings, particularly when the test smells have the same conceptual definition. Such information may bring up the common and particular aspects of test smell research in the scope of the different languages, which is subject to further research. In the sequel, we considered only test smells investigated in this work (see Section 2) and also studies with open source software.

In our work, *Duplicate Assert*, *Magic Number Test*, *Unknown Test* and *Conditional Test Logic* are the smells detected more frequently with a considerably higher number of occurrences when compared to the others. On the other hand, smells such as *Eager Test* and *Assertion Roulette*, commonly reported as the most frequent ones in empirical studies in the scope of the Java language [3, 13, 19], occurred much less frequently in our study. One possible reason is that *Conditional Test Logic*, *Magic Number Test* and *Unknown Test* have not been explored in most of those work. However, in our

study, *Eager Test* and *Assertion Roulette* have been detected only in 2-3 systems, showing that they might not be so frequent in JavaScript code. The more recent results of a study performed by Peruma et al [13] on 256 open source android applications found a moderate occurrence of the *Duplicate Assert*, *Magic Number Test*, *Unknown Test* and *Conditional Test Logic* smells, immediately following the counts for *Eager Test* and *Assertion Roulette*. The low frequency of the *Assertion Roulette* smell in JavaScript test code reported in our study may be explained by the use of clauses such "expect" and "should" from Chai that induce an auto-documentation of the assertion.

Smells such as *Mystery Guest* and *Lazy Test* have been reported as among the less frequent ones [3, 19]. This fact corroborates with our results since we found very few occurrences of those smells. Also, likewise in the study by Peruma et al. [13] we found very few occurrences of the *Mystery Guest* and *Resource Optimism* smells and also their likely co-occurrence.

Regarding co-occurrences of test smells, we found that *Eager Test* and *Lazy Test* may often occur together with a moderate correlation. In particular, Bavota et al. [3] found out that *Lazy Test* may lead to the inclusion of *Eager Test*. While co-occurrences of *Eager Test* and *Assertion Roulette* are common in other studies, in our study, they occur together in only one system, with different magnitudes, therefore, having very low correlation.

7.2.3 Test smells and Test Code Quality. Test smells have been reported in the literature as leading to defect-proneness in both production and test code and being related to low-quality design indicators [4, 6, 13, 19]. Our study found strong to very strong correlations between some test smells and standard code quality metrics when applied only to the test code. These results may indicate that test smells can directly impact code comprehension and the maintainability of test code in the JavaScript context. Further investigation is needed to confirm such claim.

7.3 Threats to Validity

Threats to external validity are concerned with the generalization of the study results. We selected 11 JavaScript projects, and due to time constraints, we impose some selection criteria that may not represent the target population. We were careful to choose criteria that represent constantly evolving projects and adopt the most popular JavaScript testing libraries. Equally, our test smell

detection tool detects only 15 types of classic test smells in 3 test libraries (Node.js' built-in assert module, Chai and Jest). However, it is open to evolution with new test smells and assertion libraries.

Threats to conclusion validity concern the relationship between independent and dependent variables. Our study used Spearman's correlation coefficient to test the relationship between test smells and quality metrics. The statistical power of our tests ($1 - \beta$) can be enhanced by increasing the number of experimental objects.

Threats to internal validity concern the facts that can impact the results. The evaluation of results of our detection tool is manual work, and this is prone to errors. We minimized this threat during the development of the tool by applying unit test suites to assess detection of each type of test smells. Note that these test suites focus only on the assertions methods from the covered testing frameworks. However, false-positive indications may occur due to the use of non-default assertions.

8 CONCLUSION

In this paper, we analyzed the occurrence of 15 classic test smells in JavaScript test suites from 11 open-source programs and which smells exist side-by-side more frequently. Furthermore, we investigate whether the test smells presence can be correlated with 12 code quality metrics. Subsequently, our results revealed that all projects have from 33% to 100% of the test suites infected, and *Duplicate Assert* is the most frequent test smell, followed by *Magic Number Test*, *Unknown Test* and *Conditional Test Logic*. Because of the incidence of different types and quantities of smells in the test cases, the results did not reveal any pattern to justify the prevalence of these smells. To facilitate our research, we developed the STEEL tool to be extensible to the needs of each project through a plugin mechanism.

Comparing our study to other similar studies, but with different languages as Java, we noticed that the incidence of smells in JavaScript behaves differently. This is due to the fact that JavaScript has distinct characteristics, such as dynamic typing or the large adoption of the most diverse types of frameworks, including unit tests. This demonstrates the need for a more in-depth study of test smells in the JavaScript language. We plan as future work to expand the catalog with JavaScript language-specific test smells.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their valuable comments. The second and third authors were supported by CNPq/Brazil (processes 315057/2018-1, 437029/2018-2, and 311215/2020-3).

REFERENCES

- [1] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian D. Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test Smell Detection Tools: A Systematic Mapping Study. In *Evaluation and Assessment in Software Engineering* (Trondheim, Norway) (EASE 2021). Association for Computing Machinery, New York, NY, USA, 170–180. <https://doi.org/10.1145/3463274.3463335>
- [2] Victor R Basili, Gianluigi Caldiera, and H Dieter Rombach. 1994. The goal question metric approach. *Encyclopedia of Software Engineering* 2 (1994), 528–532. <http://maisqual.squaring.com/wiki/index.php/TheGoalQuestionMetricApproach>
- [3] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and David Binkley. 2012. An empirical analysis of the distribution of unit test smells and their impact on software maintenance. In *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, Trento, Italy, 56–65. <https://doi.org/10.1109/ICSM.2012.6405253>
- [4] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave W. Binkley. 2015. Are test smells really harmful? An empirical study. *Empirical Software Engineering* 20, 4 (2015), 1052–1094. <https://doi.org/10.1007/s10664-014-9313-0>
- [5] Amin Milani Fard and Ali Mesbah. 2017. JavaScript: The (Un)Covered Parts. In *2017 IEEE International Conference on Software Testing, Verification and Validation, ICST 2017, Tokyo, Japan, March 13–17, 2017*. IEEE Computer Society, Tokyo, Japan, 230–240. <https://doi.org/10.1109/ICST.2017.28>
- [6] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018), 52–81. <https://doi.org/10.1016/j.jss.2017.12.013>
- [7] Geoffrey K. Gill and Chris F. Kemmerer. 1991. Cyclomatic Complexity Density and Software Maintenance Productivity. *IEEE Transactions on Software Engineering* 17, 12 (1991), 1284–1288. <https://doi.org/10.1109/32.106988>
- [8] Peter Gyimesi, Bela Vancsics, Andrea Stocco, Davood Mazinanian, Arpad Beszedes, Rudolf Ferenc, and Ali Mesbah. 2019. BugsJS: a Benchmark of JavaScript Bugs. In *2019 12th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Xi'an, China, 90–101. <https://doi.org/10.1109/ICST.2019.00019>
- [9] M. H. Halstead. 1975. Toward a theoretical basis for estimating programming effort. *Proceedings of the 1975 Annual Conference, ACM 1975 (1975)*, 222–224. <https://doi.org/10.1145/800181.810326>
- [10] Ieee. 1990. IEEE Standard Glossary of Software Engineering Terminology. *Office* 121990, 1 (1990), 1. <https://doi.org/10.1109/IEEESTD.1990.101064>
- [11] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (dec 1976), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- [12] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. 2015. JSEFT: Automated Javascript Unit Test Generation. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Graz, Austria, 1–10. <https://doi.org/10.1109/ICST.2015.7102595>
- [13] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the Distribution of Test Smells in Open Source Android Applications: An Exploratory Study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering* (Toronto, Ontario, Canada) (CASCOS '19). IBM Corp., USA, 193–202. <https://dl.acm.org/doi/10.5555/3370272.3370293>
- [14] Anthony Peruma, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. An Exploratory Study on the Refactoring of Unit Test Files in Android Applications. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*. ACM, New York, NY, USA, 350–357. <https://doi.org/10.1145/3387940.3392189>
- [15] Anthony Shehan Ayam Peruma. 2018. *What the Smell? An Empirical Investigation on the Distribution and Severity of T erity of Test Smells in Open Sour est Smells in Open Source Android Applications*. Master's thesis. Rochester Institute of Technology, New York, US.
- [16] Per Runeson and Martin Höst. 2009. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164.
- [17] Amir Saboury, Pooya Musavi, Foutse Khomh, and Giulio Antoniol. 2017. An empirical study of code smells in JavaScript projects. In *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Klagenfurt, Austria, 294–305. <https://doi.org/10.1109/SANER.2017.7884630>
- [18] Neil J. Salkind. 2018. *Exploring research*. Pearson, Harlow etc.
- [19] Davide Spadini, Fabio Palomba, Andy Zaidman, Magiel Bruntink, and Alberto Bacchelli. 2018. On the Relation of Test Smells to Software Code Quality. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Madrid, Spain, 1–12. <https://doi.org/10.1109/ICSME.2018.00010>
- [20] Davide Spadini, Martin Schvarcbacher, Ana-Maria Oprescu, Magiel Bruntink, and Alberto Bacchelli. 2020. Investigating Severity Thresholds for Test Smells. In *Proceedings of the 17th International Conference on Mining Software Repositories*. ACM, New York, NY, USA, 311–321. <https://doi.org/10.1145/3379597.3387453>
- [21] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An empirical investigation into the nature of test smells. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*. ACM, New York, NY, USA, 4–15. <https://doi.org/10.1145/2970276.2970340> arXiv:arXiv:1508.06655v1
- [22] Arash Vahabzadeh, Amin Milani Fard, and Ali Mesbah. 2015. An empirical study of bugs in test code. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, New York, NY, USA, 101–110. <https://doi.org/10.1109/ICSM.2015.7332456>
- [23] B. Van Rompaey, B. Du Bois, S. Demeyer, and M. Rieger. 2007. On The Detection of Test Smells: A Metrics-Based Approach for General Fixture and Eager Test. *IEEE Transactions on Software Engineering* 33, 12 (2007), 800–817.
- [24] Edmond VanDoren et al. 2002. Maintainability index technique for measuring program maintainability. *SEI STR Rep* (2002).